```
Start coding or generate with AI.
```

2. Perform the following operations using Python on the data sets:

Compute and display summary statistics for each feature available in the dataset. (e.g. minimum value, maximum value, mean, range, standard deviation, variance and percentiles Illustrate the feature distributions using histogram. Data cleaning, Data integration, Data transformation, Data model building (e.g. Classification)

```
import numpy as np
import pandas as pd
```

```
df = pd.read_csv('/content/bmw.csv')
df.head()
```

	model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize
0	5 Series	2014	11200	Automatic	67068	Diesel	125	57.6	2.0
1	6 Series	2018	27000	Automatic	14827	Petrol	145	42.8	2.0
2	5 Series	2016	16000	Automatic	62794	Diesel	160	51.4	3.0
3	1 Series	2017	12750	Automatic	26676	Diesel	145	72.4	1.5
4	7 Series	2014	14500	Automatic	39554	Diesel	160	50.4	3.0

Next steps: Generate code with df New interactive sheet

Summary Statistics (Min, Max, Mean, etc.)

```
df.describe()
```

	year	price	mileage	tax	mpg	engineSize
count	10781.000000	10781.000000	10781.000000	10781.000000	10781.000000	10781.000000
mean	2017.078935	22733.408867	25496.986550	131.702068	56.399035	2.167767
std	2.349038	11415.528189	25143.192559	61.510755	31.336958	0.552054
min	1996.000000	1200.000000	1.000000	0.000000	5.500000	0.000000
25%	2016.000000	14950.000000	5529.000000	135.000000	45.600000	2.000000
50%	2017.000000	20462.000000	18347.000000	145.000000	53.300000	2.000000
75%	2019.000000	27940.000000	38206.000000	145.000000	62.800000	2.000000
max	2020.000000	123456.000000	214000.000000	580.000000	470.800000	6.600000

Summary Statistics

```
df.min(numeric_only=True)
```

	0
year	1996.0
price	1200.0
mileage	1.0
tax	0.0
mpg	5.5
engineSize	0.0

dtype: float64

```
df.var(numeric_only=True)
```

	0
year	5.517980e+00
price	1.303143e+08
mileage	6.321801e+08
tax	3.783573e+03
mpg	9.820049e+02
engineSize	3.047634e-01

dtype: float64

```
df.std(numeric_only=True)
```

	0
year	2.349038
price	11415.528189
mileage	25143.192559
tax	61.510755
mpg	31.336958
engineSize	0.552054

dtype: float64

```
df.max(numeric_only=True)
```

	0
year	2020.0
price	123456.0
mileage	214000.0
tax	580.0
mpg	470.8
engineSize	6.6

dtype: float64

```
df.mean(numeric_only=True)
```

	0
year	2017.078935
price	22733.408867
mileage	25496.986550
tax	131.702068
mpg	56.399035
engineSize	2.167767

dtype: float64

```
df.max(numeric_only=True) - df.min(numeric_only=True)
```

	0
year	24.0
price	122256.0
mileage	213999.0
tax	580.0
mpg	465.3
engineSize	6.6

dtype: float64

```
df.quantile([0.25, 0.50, 0.75], numeric_only=True)
```

	year	price	mileage	tax	mpg	engineSize
0.25	2016.0	14950.0	5529.0	135.0	45.6	2.0
0.50	2017.0	20462.0	18347.0	145.0	53.3	2.0
0.75	2019.0	27940.0	38206.0	145.0	62.8	2.0

```
df.min(numeric_only=True)
df.max(numeric_only=True)
df.mean(numeric_only=True)
df.std(numeric_only=True)
df.var(numeric_only=True)
```

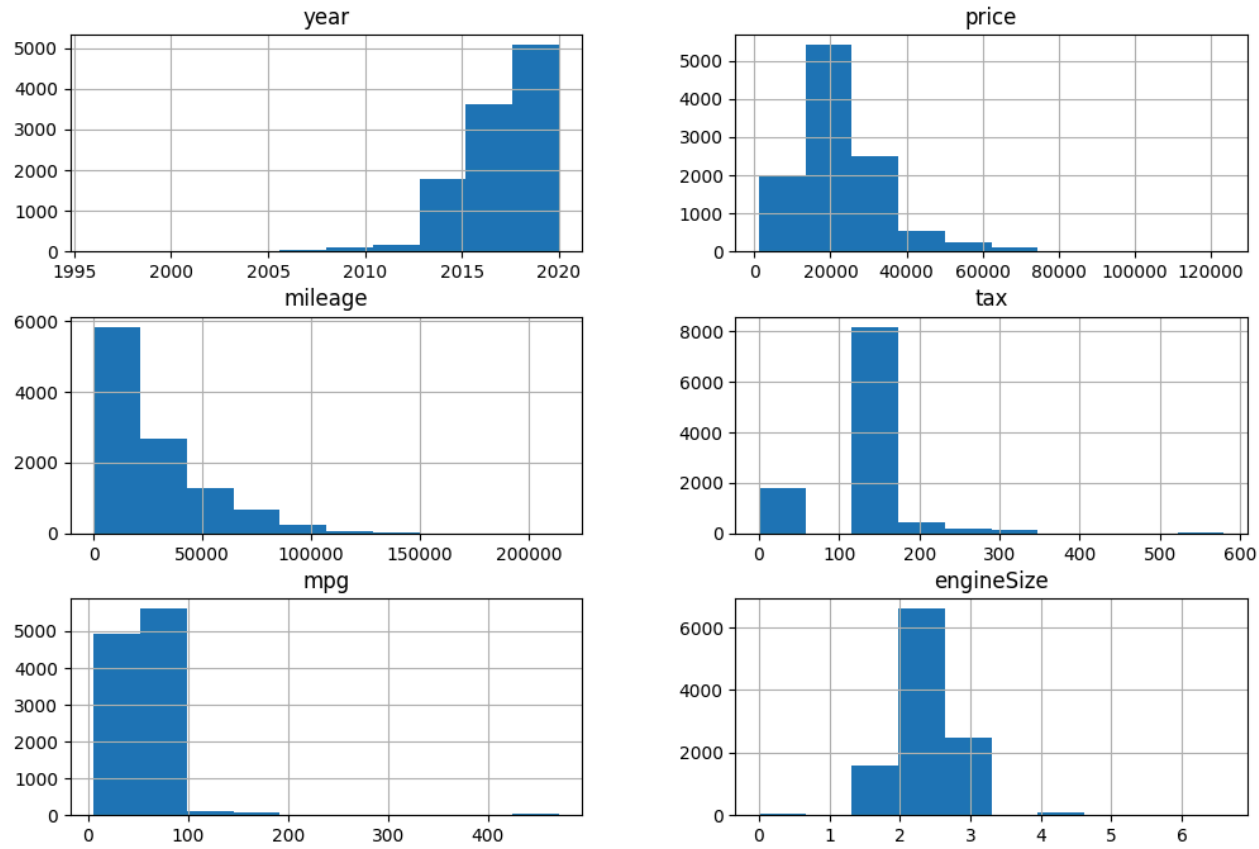
	0
year	5.517980e+00
price	1.303143e+08
mileage	6.321801e+08
tax	3.783573e+03
mpg	9.820049e+02
engineSize	3.047634e-01

dtype: float64

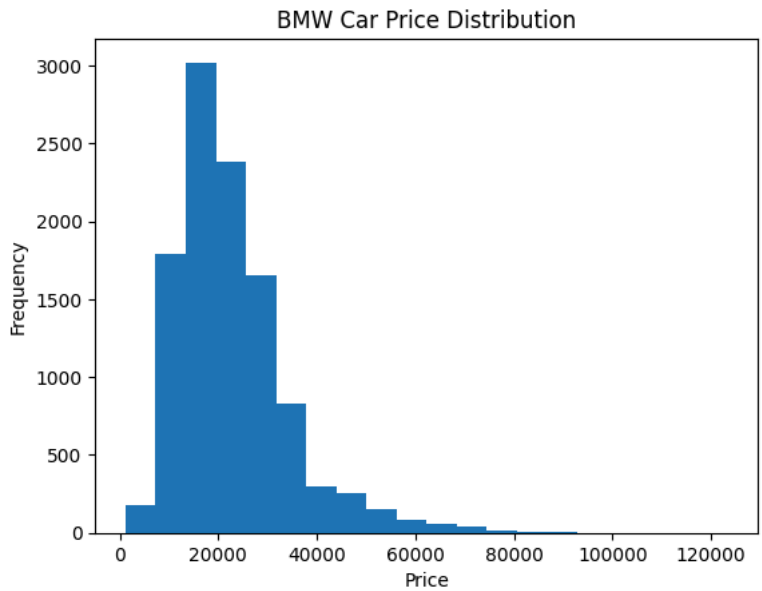
Feature Distribution using Histogram

```
import matplotlib.pyplot as plt

df.hist(figsize=(12,8))
plt.show()
```




```
plt.hist(df['price'], bins=20)
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.title('BMW Car Price Distribution')
plt.show()
```



Data Cleaning

```
df.isnull().sum()
```

	0
model	0
year	0
price	0
transmission	0
mileage	0
fuelType	0
tax	0
mpg	0